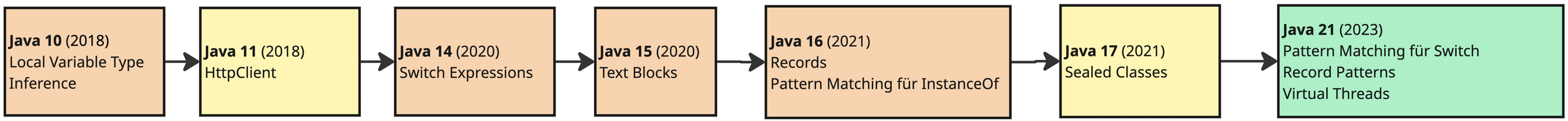
Stand der Weiterentwicklung

Historie



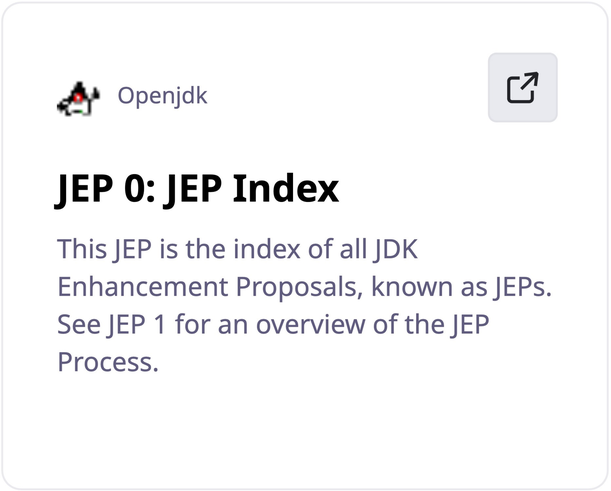
Oracle bietet noch 6 Jahre Extended Support an.

Support bis 2026 bei Oracle, bis 2027 z.B. von Bellsoft. Darüber hinaus noch Extended Support bestellbar.

Support bis 2028

JDK Enhancement Proposal (JEP)

Das JEP-Verfahren ist das strukturierte Vorgehen, um neue Features, Refactorings oder Tooling-Anpassungen in das JDK einzubringen.

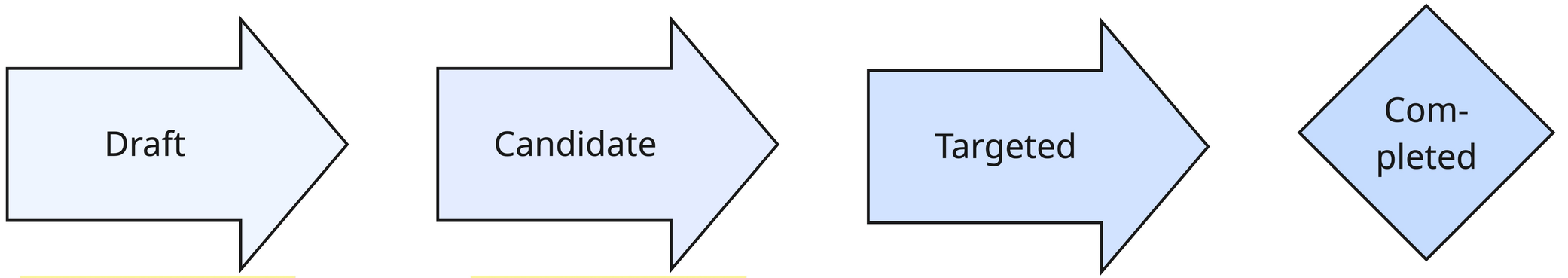


JDK Enhancement Proposal (JEP)

Eine JEP beschreibt:

- Motivation und Problemstellung
- Design-Ansatz inklusive Diskussion alternativer Ideen
- Auswirkungen auf Kompatibilität
- Performance und Sicherheit
- Tests, Referenzimplementierung und Interaktion mit Spezifikationen

JEP-Prozess



Idee wird beschrieben, Feedback wird gesammelt.

das Projektmanagement akzeptiert die JEP zur weiteren Ausarbeitung.

die JEP wird einem konkreten Release zugeordnet.

Implementierung ist in den Mainline-Code aufgenommen.

JEP-Prozess

- JEPs können zwischen Releases verschoben oder verworfen werden.
- Features erst dann einkalkulieren, wenn eine JEP tatsächlich Targeted oder Completed ist.
- JEP-Texte als Referenz für Architektur- und Coding-Guidelines nutzen (z.B. Pattern Matching, Records).
- Frühzeitig Preview- oder Early-Access-Builds testen, wenn eine JEP den eigenen Code betrifft.

Incubating vs. Preview-Feature

Incubating Feature

- Wird typischerweise über separate Module oder Tooling bereitgestellt (z.B. jdk.incubator.vector).
- API-Namen signalisieren experimentellen Status.
- Verwendung erfordert oft zusätzliche Module zur Laufzeit oder --add-modules .

Preview Feature

- Ist bereits im Standardmodulbaum, aber noch nicht endgültig.
- Aktivierung erfolgt über Compiler- und JVM-Flags (--enable-preview).
- Der Status ermöglicht Feedback, bevor die Sprach- oder API-Änderung finalisiert wird.

Beispiel: JEP 321

JEP 321: HTTP Client API

Owner	Chris Hegarty
Type	Feature
Scope	SE
Status	Closed / Delivered
Release	11
Component	core-libs / java.net
Discussion	net dash dev at openjdk dot java dot net
Effort	M
Duration	M
Relates to	JEP 110: HTTP/2 Client (Incubator) JEP 517: HTTP/3 for the HTTP Client API
Reviewed by	Alan Bateman, Brian Goetz
Endorsed by	Brian Goetz
Created	2017/06/08 11:46
Updated	2024/08/27 14:42
Issue	8181784

Die API ist seit Version 11 im Standard

Summary

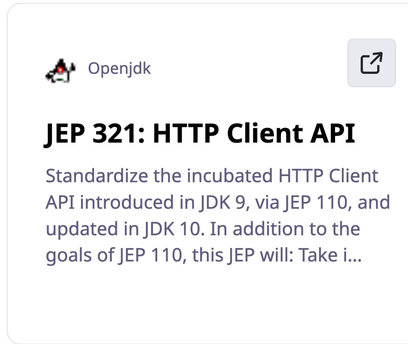
Standardize the incubated HTTP Client API introduced in JDK 9, via JEP 110, and updated in JDK 10.

Goals

In addition to the goals of JEP 110, this JEP will:

- Take into account feedback received on the incubated API,
- Provide a standardized API, in the `java.net.http` package, based upon the incubated API, and
- Remove the incubated API.

Häufig bauen große JEPs auf einer Incubator-Phase auf.



Motivation

The existing `HttpURLConnection` API and its implementation have numerous problems:

- The base `URLConnection` API was designed with multiple protocols in mind, nearly all of which are now defunct (`ftp`, `gopher`, etc.).
- The API predates HTTP/1.1 and is too abstract.
- It is hard to use, with many undocumented behaviors.
- It works in blocking mode only (i.e., one thread per request/response).
- It is very hard to maintain.

In diesem Fall sind die eigentlichen Ziele im Incubator

Goals

- Must be easy to use for common cases, including a simple blocking mode.
- Must provide notification of events such as "headers received", errors, and "response body received". This notification is not necessarily based on callbacks but can use an asynchronous mechanism like CompletableFuture.
- A simple and concise API which caters for 80-90% of application needs. This probably means a relatively small API footprint that does not necessarily expose all the capabilities of the protocol.
- Must expose all relevant aspects of the HTTP protocol request to a server, and the response from a server (headers, body, status codes, etc.).
- Must support standard and common authentication mechanisms. This will initially be limited to just Basic authentication.
- Must be able to easily set up the WebSocket handshake.
- Must support HTTP/2. (The application-level semantics of HTTP/2 are mostly the same as 1.1, though the wire protocol is completely different.)
  - Must be able to negotiate an upgrade from 1.1 to 2 (or not), or select 2 from the start.
  - Must support server push, i.e., the ability of the server to push resources to the client without an explicit request by the client.